

حقوق نشر و مالکیت فکری

COPYRIGHT & INTELLECTUAL PROPERTY NOTICE

عنوان اثر

جزوه کامل درس بازیابی اطلاعات

(Information Retrieval – University Textbook)

پدیدآورندگان

حمیدرضا نامجومنش – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

امیرحسین همتی – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

علی مجاهد – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

© حق نشر و پروانه بهره‌برداری

این اثر تحت پروانه Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) منتشر می‌شود.

✓ شما مجاز هستید:

- این اثر را به صورت غیرتجاری و بدون هیچ‌گونه تغییر، با ذکر کامل نام پدیدآورندگان باز نشر دهید.
- نسخه‌های دقیقاً مشابه را در محیط‌های آموزشی غیرانتفاعی به اشتراک بگذارید.

✗ اکیداً ممنوع است:

- هرگونه استفاده تجاری (فروش، چاپ به قصد سود، قرار دادن پشت دیوار پرداخت، و نظایر آن).
- تغییر، تلخیص، ترجمه، بازترکیب فصول یا تولید آثار مشتق شده بدون رضایت کتبی همه پدیدآورندگان.
- حذف یا مخدوش کردن اعلان‌های حق نشر و واترمارک‌های دیجیتال.

⚠ هرگونه نقض این شرایط، نقض صریح حقوق مؤلف در چارچوب قوانین ایران و کنوانسیون برن (Berne Convention) محسوب می‌شود و پیگرد قانونی به دنبال خواهد داشت.

📄 متن کامل حقوقی مجوز (انگلیسی) | خلاصه فارسی مجوز

مخزن رسمی و شناسه دیجیتال

GitHub Repository: github.com/hrnrxb/IR-Textbook

(کلیه فایل‌های اصلی، امضاها و دیجیتال و گواهی‌های زمانی در مخزن فوق نگهداری می‌شوند.)

اصالت‌سنجی و گواهی مالکیت

• تمامی نسخه‌های PDF این جزوه با واترمارک دیجیتال و فراداده کپی‌رایت محافظت شده‌اند.

• هش SHA-256 یکتای فایل‌ها در سند `HASHES.txt` مخزن ثبت شده است.

🔒 تنها نسخه‌ای که از کانال رسمی (گیت‌هاب فوق) دریافت شود، معتبر است.

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل هشتم: Evaluation in information retrieval

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

8.1 Information retrieval system evaluation

8.2 Standard test collections

8.4 Evaluation of ranked retrieval results

8.5 Assessing relevance

8.5.1 Critiques and justifications of the concept of relevance

8.6 A broader perspective: System quality and user utility

8.6.1 System issues

8.6.2 User utility

8.6.3 Refining a deployed system

8.7 Results snippets

8.8 Chapter 8 Summary

۸.۱ چرا بازیابی اطلاعات یک علم "آزمایشگاهی" است؟

آشپزخانه مهندسی: کدام ادویه جواب می‌دهد؟

تا اینجا درس، ما مثل آشپزهایی بودیم که انواع مواد اولیه را یاد گرفتیم: ریشه‌یابی (Stemming)، حذف کلمات توقف (Stop Lists)، وزن‌دهی tf-idf و مدل‌های برداری. اما سوال اصلی اینجا است: «از کجا بفهمیم کدام ترکیب خوشمزه‌ترین غذا را می‌سازد؟»

آیا باید حتماً کلمات را ریشه‌یابی کنیم؟ آیا استفاده از idf همیشه نتیجه را بهتر می‌کند؟ در ریاضیات، $2 + 2$ همیشه ۴ می‌شود. اما در بازیابی اطلاعات، پاسخ این سوالات **بستگی دارد**. به همین دلیل است که تأکید می‌شود IR یک رشته کاملاً تجربی (Highly Empirical) است. ما نمی‌توانیم پشت میز بنشینیم و اثبات کنیم روش X بهتر است؛ باید آن را روی داده‌های واقعی تست کنیم.

برای اثبات برتری یک تکنیک جدید، ما نیاز به ارزیابی دقیق و کامل روی مجموعه‌های اسناد نماینده (Representative Document Collections) داریم. دقیقاً همان کاری که تیم‌های مهندسی در شرکت‌های بزرگ انجام می‌دهند.

📺 مثال واقعی: وسواسِ Netflix در ارزیابی تجربی

شاید فکر کنید Netflix فقط فیلم‌ها را به شما پیشنهاد می‌دهد. اما آن‌ها حتی روی تصویر کاور (Thumbnail) فیلم‌ها هم ارزیابی تجربی انجام می‌دهند!

زمانی که سریال *Stranger Things* منتشر شد، منتقلیکس یک تصویر ثابت را به همه نشان نداد. آن‌ها الگوریتم‌های مختلفی را تست کردند که به گروه‌های مختلف کاربران، تصاویر متفاوتی نشان می‌داد (تست A/B). نتیجه؟ آن‌ها متوجه شدند که اگر تصویر شامل چهره‌ای با احساس "ترس" باشد، نرخ کلیک به شدت بالاتر می‌رود.

این یعنی بازیابی اطلاعات در عمل! آن‌ها حدس نزدند؛ آن‌ها **ارزیابی** کردند. در این فصل یاد می‌گیریم چطور این تست‌ها را طراحی کنیم.

در ادامه این فصل، ما متدولوژی‌های رسمی برای ارزیابی نتایج را بررسی می‌کنیم (بخش ۸.۳ و ۸.۴) و یاد می‌گیریم چطور مجموعه‌های تست قابل اعتماد بسازیم (بخش ۸.۲ و ۸.۵). اما قبل از ورود به ریاضیاتِ ارزیابی، باید یک قدم به عقب برگردیم و به هدف نهایی فکر کنیم.

😊 مفهوم "شادی کاربر" (User Happiness)

همه فرمول‌ها و متریک‌هایی که خواهیم خواند، تنها یک هدف دارند: تقریب‌زدنِ مفهومی به نام **سودمندی کاربر (User Utility)**. به زبان ساده‌تر: آیا کاربر خوشحال شد؟

شادی کاربر به فاکتورهای مختلفی بستگی دارد:

- **سرعت پاسخ‌دهی (Speed):** سیستم چقدر سریع جواب داد؟
- **مرتبط بودن نتایج (Relevance):** آیا جواب‌ها به درد می‌خوردند؟
- **رابط کاربری (UI):** آیا ظاهر سیستم تمیز و واضح است؟

🚀 آمازون: جنگِ سرعت در برابر کیفیت

شرکت Amazon سال‌ها پیش تحقیقی منتشر کرد که نشان می‌داد هر ۱۰۰ میلی‌ثانیه تأخیر در لود شدن صفحه، باعث کاهش ۱٪ در فروش می‌شود. سرعت مهم است!

اما بیایید منطقی باشیم: اگر آمازون با سرعت نور (۰ ثانیه) نتیجه را برگرداند، اما وقتی شما "کفش نایک" را جستجو می‌کنید به شما "موز" نشان دهد، آیا خوشحال می‌شوید؟ قطعاً نه.

"Blindingly fast, useless answers do not make a user happy" (پاسخ‌های کورکننده سریع اما بی‌استفاده، کاربر را خوشحال نمی‌کنند). پس فرض معقول ما این است که **Relevance** (ارتباط معنایی) مهم‌ترین فاکتور است، هرچند تنها فاکتور نیست.

البته گاهی اوقات درک کاربر از کیفیت، با متریک‌های مهندسی ما یکی نیست. طراحی رابط کاربری (UI)، چیدمان و حتی **اسنیپت‌ها (Snippets)** - آن چند خط توضیحی که زیر لینک آبی در گوگل می‌بینید - تأثیر عظیمی بر رضایت کاربر دارند (بخش ۸.۷)، حتی اگر رتبه‌بندی نتایج یکسان باشد.

🎨 گوگل و ۴۱ طیف رنگ آبی

برای اینکه بفهمید چقدر مسائلی غیر از "الگوریتم جستجو" روی شادی کاربر تأثیر دارد: **ماريسا مایر** (زمانی که در گوگل بود)، دستور داد **۴۱ طیف مختلف از رنگ آبی** را برای لینک‌های تبلیغاتی تست کنند تا ببینند کاربران روی کدام رنگ بیشتر کلیک می‌کنند.

این ربطی به کیفیت نتایج نداشت، اما مستقیماً روی تعامل کاربر (و درآمد گوگل!) تأثیر داشت. این نشان می‌دهد که "ارزیابی" در دنیای واقعی، ترکیبی پیچیده از الگوریتم، روانشناسی و تجربه کاربری است.

در این فصل، ما روی بخش **الگوریتمی** تمرکز خواهیم کرد: چگونه "ارتباط" (Relevance) را اندازه بگیریم تا مطمئن شویم موتور جستجوی ما، صرف‌نظر از رنگ دکمه‌هایش، محتوای درستی را پیدا می‌کند.

۸.۱ آزمایشگاه ارزیابی اطلاعات: چگونه به گوگل نمره دهیم؟ 📄

🏠 سه‌پایه مقدس ارزیابی (The Standard Test Collection)

فرض کنید شما مهندس ارشد در **Bing** یا **Spotify** هستید و ادعا می‌کنید الگوریتم جدیدی نوشته‌اید که نتایج بهتری می‌دهد. مدیرتان حرف شما را قبول نمی‌کند مگر اینکه "اثبات" کنید. در دنیای IR، ما نمی‌توانیم به احساسمان اعتماد کنیم. ما به یک "میز تشریح" استاندارد نیاز داریم که از سه جزء حیاتی تشکیل شده است:

1. 📁 **انبار اسناد (Document Collection)**: یک مجموعه ثابت از اسناد (مثلاً ۱ میلیون صفحه وب یا ۵۰ هزار مقاله پزشکی).
2. 🗨️ **مجموعه نیازهای اطلاعاتی (Information Needs)**: لیستی از سوالات یا دغدغه‌های واقعی کاربران که به زبان ماشین (Query) ترجمه شده‌اند.
3. ✅ **کلید آزمون (Relevance Judgments)**: بخش سخت ماجرا! برای هر جفتِ «پرسمان-سند»، یک انسان (یا داور) مشخص کرده که آیا این سند "مرتبط" است یا خیر. به این قضاوت‌ها، **استاندارد طلایی (Gold Standard)** یا **Ground Truth** می‌گوییم.

شاید بپرسید چرا قضاوت "مرتبط بودن" اینقدر پیچیده است؟ چرا فقط نمی‌گردیم ببینیم کلمات پرسمان در متن هست یا نه؟

🤖 تفاوت ظریف و حیاتی: "نیاز اطلاعاتی" در برابر "پرسمان"

اینجا جایی است که سیستم‌های آماتور از حرفه‌ای جدا می‌شوند. کاربری را تصور کنید که کلمه "**Python**" را در گوگل تایپ می‌کند.

- آیا او یک برنامه‌نویس است که دنبال داکيومنتیشن زبان پایتون می‌گردد؟ 📖
- یا یک علاقه‌مند به خزندگان است که می‌خواهد یک مار پایتون خانگی بخرد؟ 🐍

از روی یک کلمه (Query)، سیستم گیج می‌شود. اما کاربر در ذهن خود یک **نیاز اطلاعاتی (Information Need)** شفاف دارد.

مثال : فرض کنید نیاز اطلاعاتی کاربر این است: "اطلاعاتی درباره اینکه آیا شراب قرمز در کاهش ریسک حمله قلبی موثرتر از شراب سفید است؟"

کاربر این را به یک پرسمان کلیدواژه‌ای تبدیل می‌کند: wine AND red AND white AND heart AND attack AND effective.

حالا اگر سندی داشته باشیم که شامل تمام این کلمات باشد اما درباره "تأثیر مخرب الکل بر قلب" صحبت کند، از نظر سیستم (تطبیق کلمات) عالی است، اما از نظر کاربر (نیاز اطلاعاتی) **غیرمرتبط (Non-relevant)** است.

بنابراین، در ارزیابی استاندارد، ما سند را بر اساس اینکه آیا **نیاز کاربر را رفع می‌کند** قضاوت می‌کنیم، نه اینکه صرفاً کلماتش را دارد. فعلاً هم فرض می‌کنیم دنیا سیاه و سفید است: سند یا مرتبط است (۱) یا نیست (۰). (هرچند در واقعیت طیفی از خاکستری است).

📌 قانون ۵۰: چرا یک پرسمان کافی نیست؟

اگر الگوریتم شما روی پرسمان "Python" عالی کار کرد، آیا الگوریتم خوبی است؟ خیر. نتایج جستجو نویز زیادی دارند. ممکن است روی یک پرسمان شانسی عالی باشید و روی دیگری فاجعه. طبق تجربه مهندسی (Rule of thumb)، برای اینکه ارزیابی شما از نظر آماری قابل اتکا باشد، باید میانگین عملکرد را روی **حداقل ۵۰ نیاز اطلاعاتی مختلف** محاسبه کنید. درست مثل اینکه نمی‌توانید کیفیت یک رستوران را فقط با خوردن یک سیب‌زمینی سرخ‌کرده بسنجید؛ باید منوی کامل را تست کنید!

⚠️ تله‌ی مرگبار: تقلب در امتحان (Overfitting)

این مهم‌ترین درس برای کسانی است که می‌خواهند در حوزه Data Science یا AI کار کنند (مثل مسابقات Kaggle). بسیاری از سیستم‌های IR پارامترهایی دارند که قابل تنظیم‌اند (Tuning). یک اشتباه رایج و نابخشودنی این است که پارامترها را طوری تنظیم کنید که روی "مجموعه تست" بهترین جواب را بدهند. این دقیقاً مثل این است که دانش‌آموزی سوالات امتحان نهایی را بدزدد، جواب‌ها را حفظ کند و نمره ۲۰ بگیرد. این دانش‌آموز باسواد نیست، فقط تقلب کرده است!

راه درست چیست؟

باید داده‌ها را جدا کنید:

- **Development Test Collection (مجموعه توسعه):** پارامترهای سیستم را اینجا تنظیم و دستکاری کنید (چرک‌نویس).
- **Test Collection (مجموعه تست نهایی):** وقتی سیستم نهایی شد، فقط یک بار روی این مجموعه اجرا کنید تا نمره واقعی و بدون سوگیری (Unbiased) مشخص شود.

در بخش‌های بعد، یاد می‌گیریم وقتی این "کلید آزمون" را داریم، دقیقاً چه نمره‌ای (Precision, Recall, etc) به سیستم بدهیم.

۸.۲ مجموعه‌های تست استاندارد: زمین بازی برای سنجش کارایی سیستم‌های IR

آیا تا به حال به این فکر کرده‌اید که چطور شرکت‌هایی مثل Microsoft (با Bing) یا تیم‌های پژوهشی دانشگاهی می‌توانند ادعا کنند که الگوریتم جدید جستجوی آن‌ها بهتر از بقیه کار می‌کند؟ پاسخ در اینجا است: مجموعه‌های تست استاندارد.

🔦 نقش مجموعه‌های تست در دنیای واقعی

این مجموعه‌ها، شبیه به «زمین بازی» یا «بنچ‌مارک» های متمرکز هستند که شامل اسناد (Documents)، پرسمان‌ها (Queries/Topics)، و مهم‌تر از همه، قضاوت‌های مرتبط بودن (Relevance Judgments) می‌باشند. این قضاوت‌ها، که توسط انسان‌ها انجام شده‌اند، حکم پاسخ صحیح (Ground Truth) را دارند. وجود این پاسخ صحیح به ما امکان می‌دهد تا عملکرد هر الگوریتم جدید بازاریابی اطلاعات را با دقت کمی و قابل تکرار اندازه‌گیری کنیم و بگوییم روش A دقیقاً چقدر از روش B بهتر است.

ما در اینجا به طور خاص بر مجموعه‌های تست برای ارزیابی سیستم‌های بازاریابی اطلاعات آزاد (Ad Hoc IR) متمرکز هستیم، اما به چند مجموعه مهم در حوزه طبقه‌بندی متنی (Text Classification) نیز اشاره می‌کنیم.

• Cranfield (کرنفیلد):

- اهمیت تاریخی: این مجموعه پیشگام، در اواخر دهه ۱۹۵۰ در انگلستان جمع‌آوری شد و اولین مجموعه‌ای بود که اندازه‌گیری‌های کمی دقیق کارایی IR را ممکن ساخت.
- محتوا: ۱۳۹۸ چکیده مقالات ژورنال آیرودینامیک و ۲۲۵ پرسمان.
- ویژگی منحصر به فرد: دارای قضاوت‌های مرتبط بودن کامل و جامع (Exhaustive Relevance Judgments) برای همه جفت‌های (پرسمان، سند).
- کاربرد امروزی: به دلیل اندازه بسیار کوچک، فقط برای آزمایش‌های مقدماتی (Pilot Experiments) مناسب است.

• TREC (کنفرانس بازاریابی متنی):

- پیشگام وب: از سال ۱۹۹۲ توسط NIST (مؤسسه ملی استانداردها و فناوری آمریکا) اجرا شده و بزرگترین بستر تست برای IR است.
- مجموعه اصلی: مشهورترین زیرمجموعه‌ها، مربوط به مسیر TREC Ad Hoc (بین سال‌های ۱۹۹۲ تا ۱۹۹۹) هستند که در مجموع شامل ۱.۸۹ میلیون سند (اغلب مقالات خبری) و قضاوت برای ۴۵۰ نیاز اطلاعاتی (Topics) می‌باشند.
- زیرمجموعه توصیه‌شده: TREC 6-8 که ۱۵۰ نیاز اطلاعاتی بر روی حدود ۵۲۸,۰۰۰ مقاله خبری دارد، اغلب به عنوان بهترین زیرمجموعه برای پژوهش‌های آتی توصیه می‌شود.
- چالش مقیاس و راه‌حل (Pooling): از آنجایی که این مجموعه‌ها بسیار بزرگ هستند، ارزیابی مرتبط بودن همه اسناد غیرممکن است. بنابراین، ارزیابان NIST قضاوت‌های مرتبط بودن را فقط برای اسنادی انجام دادند که جزو نتایج Top k (مثلاً ۱۰۰۰ سند برتر) سیستم‌های شرکت‌کننده در آن ارزیابی خاص بودند. این روش هوشمندانه به نام Pooling، ارزیابی کارآمد مجموعه‌های عظیم را ممکن می‌سازد.

• GOV2 (مجموعه صفحات وب دولتی):

- محتوا: شامل ۲۵ میلیون صفحه از وب (محتوای وبسایت‌های دولتی آمریکا).
- اهمیت: این مجموعه در زمان خود از هر مجموعه دیگری که در دسترس پژوهشگران بود، به مراتب بزرگی بزرگتر بود و همچنان بزرگترین مجموعه وب است که به راحتی برای اهداف تحقیقاتی در دسترس است.
- نکته کلیدی: با وجود بزرگی، اندازه GOV2 هنوز بیش از دو مرتبه بزرگی (صد برابر) کوچکتر از مجموعه‌های سند ایندکس‌شده توسط موتورهای جستجوی بزرگ وب (مانند Google Search یا Baidu) است.

• NTCIR و CLEF (بازیابی میان‌زبانی):

تصور کنید شما در UNICEF (صندوق کودکان سازمان ملل) کار می‌کنید و باید گزارشی در مورد واکسیناسیون در آسیای شرقی تهیه کنید. شما پرسمان خود را به زبان انگلیسی وارد می‌کنید، اما سیستم باید بتواند اسناد مرتبط را از مقالات خبری نوشته‌شده به زبان‌های ژاپنی، چینی یا کره‌ای پیدا کند. این وظیفه، هسته بازیابی اطلاعات میان‌زبانی (Cross-Language Information Retrieval) است.

▪ NTCIR (ژاپن): مجموعه‌ای مشابه TREC که بر روی زبان‌های آسیای شرقی و بازیابی اطلاعات میان‌زبانی متمرکز شده است.

▪ CLEF (اروپا): بر روی زبان‌های اروپایی و بازیابی اطلاعات میان‌زبانی تمرکز دارد.

• Reuters-21578 و RCV1 (طبقه‌بندی متنی):

▪ حوزه کاربرد: این مجموعه‌ها برای ارزیابی سیستم‌های طبقه‌بندی متنی، جایی که هدف تعیین دسته‌بندی موضوعی سند است (مانند "سیاست"، "ورزش" یا "اقتصاد")، استفاده می‌شوند.

▪ Reuters-21578: مجموعه‌ای کلاسیک از ۲۱۵۷۸ مقاله خبری.

▪ RCV1 (Reuters Corpus Volume 1): این مجموعه بسیار بزرگتر، شامل ۸۰۶,۷۹۱ سند است و به دلیل مقیاس و برچسب‌گذاری غنی، پایه بهتری برای پژوهش‌های مدرن (مبتنی بر یادگیری عمیق) در آینده فراهم می‌کند. (به فصل ۴، مراجعه کنید)

• Newsgroups 20 (۲۰ گروه خبری):

▪ محتوا: مجموعه‌ای پرکاربرد که توسط Ken Lang جمع‌آوری شده است. شامل ۱۰۰۰ مقاله از هر یک از ۲۰ گروه خبری Usenet (در مجموع ۱۸۹۴۱ مقاله پس از حذف تکراری‌ها).

▪ کاربرد: نام گروه خبری (مانند alt.atheism یا comp.graphics) به عنوان دسته‌بندی سند در نظر گرفته می‌شود و به طور گسترده در ارزیابی الگوریتم‌های طبقه‌بندی و خوشه‌بندی استفاده می‌شود.

نکته پایانی: در حوزه پژوهش، مجموعه‌های تست به ما زبان مشترکی برای مقایسه می‌دهند. تفاوت بین Cranfield که ارزیابی کامل دارد با TREC که از تکنیک Pooling استفاده می‌کند، نشان‌دهنده تکامل این حوزه از مقیاس آزمایشگاهی به مقیاس وب است.

۸.۳ ارزیابی مجموعه‌های بازیابی بدون رتبه

اثر بخشی سیستم‌های بازیابی اطلاعات (IR) را چطور اندازه‌گیری کنیم؟ برای حالت ساده‌ای که سیستم صرفاً مجموعه‌ای از اسناد را برای یک پرسمان بازمی‌گرداند (بدون رتبه‌بندی)، دو معیار اساسی برای سنجش کارایی تعریف می‌شوند: **Precision (دقت)** و **Recall (بازخوانی)**. در بخش‌های بعدی این مفاهیم برای حالت بازیابی رتبه‌دار (Ranked Retrieval) نیز گسترش می‌یابند.

• **Precision (P) - دقت:** کسری از کل اسناد بازیابی‌شده که واقعاً مرتبط هستند.

$$\text{Precision} = \frac{\text{relevant items retrieved}}{\text{retrieved items}} = P(\text{relevant}|\text{retrieved}) \quad (8.1)$$

• **Recall (R) - بازخوانی:** کسری از کل اسناد مرتبط موجود که توسط سیستم بازیابی شده‌اند.

$$\text{Recall} = \frac{\text{relevant items retrieved}}{\text{relevant items}} = P(\text{retrieved}|\text{relevant}) \quad (8.2)$$

این مفاهیم با بررسی جدول رخداد (Contingency Table) زیر، که تمام حالات ممکن را بر اساس قضاوت واقعی (مرتبط/نامرتبط) و تصمیم سیستم (بازیابی‌شده/بازیابی‌نشده) نشان می‌دهد، شفاف‌تر می‌شوند:

واقعاً نامرتب (Nonrelevant)	واقعاً مرتب (Relevant)	
fp (False Positives)	tp (True Positives)	توسط سیستم بازیابی شد (Retrieved)
tn (True Negatives)	fn (False Negatives)	توسط سیستم بازیابی نشد (Not Retrieved)

فرمول‌ها بر اساس جدول (۸.۴):

$$P = \frac{tp}{tp + fp} \quad \text{and} \quad R = \frac{tp}{tp + fn}$$

چالش معیار دقت (Accuracy) در بازیابی اطلاعات

یک معیار بدیهی که در یادگیری ماشین برای ارزیابی طبقه‌بندی (Classification) استفاده می‌شود، دقت (Accuracy) است: کسری از کل طبقه‌بندی‌های درست که شامل $(tp + tn)$ می‌شود:

$$\text{Accuracy} = \frac{tp + tn}{tp + fp + fn + tn}$$

چرا Accuracy معیار مناسبی برای IR نیست؟

در تقریباً تمام سناریوهای بازیابی اطلاعات، توزیع داده‌ها به شدت نامتوازن (Skewed) است: معمولاً بیش از ۹۹.۹٪ از اسناد موجود، نسبت به یک پرسمان خاص، نامرتب (tn) هستند.

مثال واقعی (پژوهش پزشکی): یک محقق در مرکز تحقیقات بیماری‌های خاص در ایران در حال جستجو در یک مجموعه ۱,۰۰۰,۰۰۰ مقاله پزشکی برای یافتن مقالات مربوط به یک سندروم فوق‌العاده نادر است. فرض کنید تنها ۵ مقاله واقعاً مرتبط هستند.

- اگر سیستمی بسازید که صرفاً تصمیم بگیرد هیچ مقاله‌ای مرتبط نیست و هیچ چیزی را بازنگرداند (

$tp = 0, fp = 0$)، این سیستم به $tn \approx 999,995$ دست می‌آید.

- Accuracy سیستم می‌شود: $\frac{0+999,995}{1,000,000} \approx 99.9995\%$

این دقت به طور کاذب بالاست، اما سیستم عملاً شکست خورده است چون تمام ۵ مقاله حیاتی (که $fn = 5$ هستند) را از دست داده و برای کاربر کاملاً بی‌فایده است. کاربران IR همیشه انتظار دارند حداقل مقداری اطلاعات مفید ببینند. بنابراین، Precision و Recall با تمرکز بر روی مثبت‌های درست (tp) ، ارزیابی را متمرکزتر و با معنی (meaningful) می‌کنند.

مبادله (Trade-off) میان Precision و Recall

این دو معیار اغلب در تضاد با هم هستند. شما همیشه می‌توانید با بازیابی تمام اسناد مجموعه، Recall را به ۱۰۰٪ برسانید (اما Precision فاجعه‌بار خواهد بود). به همین دلیل، اینکه کدام معیار مهم‌تر است، بستگی به هدف جستجوگر دارد:

۱. اولویت P (دقت): کاربران وب. کاربری که در دیجی‌کالا به دنبال بهترین راهنمای خرید یک لپ‌تاپ است، می‌خواهد ۱۰ نتیجه اولش همه دقیق و مرتبط باشند. او علاقه‌ای به دیدن هزاران سند مرتبط (Low Recall tolerance) ندارد و صرفاً نتایج دقیق را در بالای صفحه می‌خواهد.

۲. اولویت R (بازخوانی): تحلیلگران و حقوقدانان. یک وکیل یا مشاور حقوقی در شرکت حقوقی کاردان که به دنبال هر نوع سابقه یا اشاره‌ای به یک قانون خاص در هزاران سند بایگانی‌شده است، باید حتماً هر مدرک مرتبط را پیدا کند. او حاضر

است نتایج زیادی را که نامرتب هستند (Low Precision tolerance) بررسی کند، به شرطی که حتی یک سند حیاتی از دست نرود (High Recall is critical).

F-measure: میانگین هارمونیک وزن دار

برای داشتن یک معیار واحد که Precision و Recall را با هم در نظر بگیرد و رابطه مبادله آن‌ها را منعکس کند، از **F-measure** استفاده می‌شود. این معیار، **میانگین هارمونیک وزن دار** این دو مقدار است:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \quad \text{or} \quad F = \frac{(1 + \beta^2)PR}{\beta^2 P + R} \quad (8.5)$$
$$\text{where } \beta^2 = \frac{1 - \alpha}{\alpha}$$

در رایج‌ترین حالت، از F_1 استفاده می‌شود که در آن Precision و Recall به صورت متوازن (Equal Weighting) وزن‌دهی شده‌اند. این به معنی $\alpha = 1/2$ و $\beta = 1$ است و فرمول آن به شکل زیر ساده می‌شود:

$$F_1 = \frac{2PR}{P + R} \quad (8.6)$$

توجه داشته باشید که $\beta < 1$ (مثلاً $\beta = 0.5$) بر **Precision** تأکید می‌کند و $\beta > 1$ (مثلاً $\beta = 3$) بر **Recall** تأکید می‌کند. انتخاب β باید مطابق با اولویت‌های کاربردی (مثل مثال‌های بالا) باشد.

نقش حیاتی میانگین هارمونیک

چرا از میانگین هارمونیک به جای میانگین حسابی (Arithmetic Mean) ساده استفاده می‌کنیم؟

💡 دلیل علمی: استبداد حداقل (Tyranny of the Minimum)

میانگین حسابی در شرایطی که یکی از دو مقدار (P یا R) نزدیک به صفر باشد، به شدت گمراه‌کننده است. چون همیشه می‌توان Recall را ۱۰۰٪ کرد، میانگین حسابی همیشه به سمت ۵۰٪ میل می‌کند (حتی اگر Precision صفر باشد!).

میانگین هارمونیک: این معیار همواره کمتر یا مساوی با میانگین حسابی و هندسی است. ویژگی کلیدی آن این است که **بسیار به حداقل دو مقدار نزدیک است**. به عبارت دیگر، مقدار نهایی F-measure تحت **استبداد مقدار حداقل** قرار دارد.

برای مثال بالا که $R = 100\%$ و $P = 0.01\%$ بود:

- میانگین حسابی: 50.005% (کاملاً گمراه‌کننده)
- میانگین هارمونیک (F_1): $0.02\% \approx$ (نزدیک به 0.01% و واقع‌بینانه)

بنابراین، میانگین هارمونیک تضمین می‌کند که یک سیستم برای کسب امتیاز بالا در F-measure، باید در **هر دو** معیار Precision و Recall، عملکرد مناسبی داشته باشد.

این دو معیار و F-measure، ستون فقرات ارزیابی کارایی در بازیابی اطلاعات هستند و در فصل‌های آینده برای ارزیابی نتایج **رتبه‌دار**، توسعه خواهند یافت.

```
In [6]: import numpy as np
import matplotlib.pyplot as plt

def calculate_ir_metrics(tp, fp, fn, tn):
```

محاسبه معیارهای اصلی ارزیابی سیستم‌های بازیابی اطلاعات

پارامترها:

tp: تعداد اسناد مرتبطی که به درستی بازیابی شده‌اند

fp: تعداد اسناد نامرتبّی که به اشتباه بازیابی شده‌اند

fn: تعداد اسناد مرتبطی که به اشتباه بازیابی نشده‌اند

tn: تعداد اسناد نامرتبّی که به درستی بازیابی نشده‌اند

خروجی:

precision, recall, accuracy, f1 دیکشنری شامل مقادیر

نسبت اسناد مرتبط در میان اسناد بازیابی شده: precision محاسبه

precision = tp / (tp + fp) **if** (tp + fp) > 0 **else** 0

نسبت اسناد مرتبط بازیابی شده در میان کل اسناد مرتبط: recall محاسبه

recall = tp / (tp + fn) **if** (tp + fn) > 0 **else** 0

نسبت کل تصمیمات درست به کل تصمیمات: accuracy محاسبه

accuracy = (tp + tn) / (tp + fp + fn + tn) **if** (tp + fp + fn + tn) > 0 **else** 0

precision و recall میانگین هارمونیک: F1-score محاسبه

f1 = 2 * precision * recall / (precision + recall) **if** (precision + recall) > 0 **else** 0

```
return {  
    'precision': precision,  
    'recall': recall,  
    'accuracy': accuracy,  
    'f1': f1  
}
```

def calculate_f_beta(precision, recall, beta):

"""

beta برای مقادیر مختلف F-beta score محاسبه

پارامترها:

precision: مقدار

recall: مقدار

beta: precision (beta > 1: اولویت با recall, beta < 1: اولویت با precision) وزن نسبت به

خروجی:

مقدار F-beta score

"""

beta_squared = beta ** 2

return (1 + beta_squared) * precision * recall / (beta_squared * precision + recall)

def compare_means(precision, recall):

"""

"Tyranny of the Minimum" مقایسه میانگین‌های مختلف (حسابی، هندسی، هارمونیک) برای نمایش مفهوم

پارامترها:

precision: مقدار

recall: مقدار

خروجی:

دیکشنری شامل مقادیر میانگین‌های مختلف

"""

میانگین حسابی

arithmetic_mean = (precision + recall) / 2

میانگین هندسی

geometric_mean = np.sqrt(precision * recall)

میانگین هارمونیک (F1-score)

harmonic_mean = 2 * precision * recall / (precision + recall) **if** (precision + recall) > 0 **else** 0

```
return {  
    'arithmetic': arithmetic_mean,  
    'geometric': geometric_mean,  
    'harmonic': harmonic_mean  
}
```

def plot_means_comparison():

"""

"Tyranny of the Minimum" رسم نمودار مقایسه‌ای بین میانگین‌های مختلف برای نمایش مفهوم

"""

precision و recall ایجاد مقادیر مختلف

precision_values = np.linspace(0.01, 1, 100)

recall_values = np.ones_like(precision_values) *همیشه 1 است recall فرض می‌کنیم* #

محاسبه میانگین‌ها

arithmetic_means = (precision_values + recall_values) / 2

geometric_means = np.sqrt(precision_values * recall_values)

harmonic_means = 2 * precision_values * recall_values / (precision_values + recall_values)

رسم نمودار

plt.figure(figsize=(10, 6))

plt.plot(precision_values, arithmetic_means, label='Arithmetic Mean', linewidth=2)

plt.plot(precision_values, geometric_means, label='Geometric Mean', linewidth=2)

plt.plot(precision_values, harmonic_means, label='Harmonic Mean (F1)', linewidth=2)

plt.xlabel('Precision')

plt.ylabel('Mean Value')

plt.title('Comparison of Different Means when Recall = 1')

plt.legend()

plt.grid(True)

plt.show()

def plot_f_beta_scores():

"""

beta برای مقادیر مختلف F-beta scores رسم نمودار

"""

precision و recall ایجاد مقادیر مختلف

precision_values = np.linspace(0.1, 1, 50)

```

recall_values = np.linspace(0.1, 1, 50)

# ایجاد شبکه برای رسم نمودار سه بعدی
P, R = np.meshgrid(precision_values, recall_values)

# برای beta مقادیر مختلف F-beta محاسبه
betas = [0.5, 1, 2]
fig = plt.figure(figsize=(15, 5))

for i, beta in enumerate(betas):
    ax = fig.add_subplot(1, 3, i+1, projection='3d')
    beta_squared = beta ** 2
    F = (1 + beta_squared) * P * R / (beta_squared * P + R)

    ax.plot_surface(P, R, F, cmap='viridis')
    ax.set_xlabel('Precision')
    ax.set_ylabel('Recall')
    ax.set_zlabel(f'F{beta} Score')
    ax.set_title(f'F{beta} Score Surface')

plt.tight_layout()
plt.show()

# مثال 1: سناریوی متعادل (مجموعه 1000 سند با 20 سند مرتبط)
print("=" * 60)
print("مثال 1: سناریوی متعادل")
print("=" * 60)
tp1, fp1, fn1, tn1 = 8, 10, 12, 970 # 8 سند مرتبط از دست رفته 12 سند نامرتب بازبایی شده،
metrics1 = calculate_ir_metrics(tp1, fp1, fn1, tn1)
print(f"Precision: {metrics1['precision']:.4f}")
print(f"Recall: {metrics1['recall']:.4f}")
print(f"Accuracy: {metrics1['accuracy']:.4f}")
print(f"F1-Score: {metrics1['f1']:.4f}")

# مقایسه میانگین‌ها برای مثال 1
means1 = compare_means(metrics1['precision'], metrics1['recall'])
print(f"\nمقایسه میانگین‌ها:")
print(f"میانگین حسابی: {means1['arithmetic']:.4f}")
print(f"میانگین هندسی: {means1['geometric']:.4f}")
print(f"میانگین هارمونیک: {means1['harmonic']:.4f}")

# مثال 2: سناریوی نامتوازن (مجموعه 1,000,000 سند با 5 سند مرتبط)
print("\n" + "=" * 60)
print("Accuracy مشکل معیار) مثال 2: سناریوی نامتوازن")
print("=" * 60)
tp2, fp2, fn2, tn2 = 0, 0, 5, 999995 # بازبایی نکرده
metrics2 = calculate_ir_metrics(tp2, fp2, fn2, tn2)
print(f"Precision: {metrics2['precision']:.4f}")
print(f"Recall: {metrics2['recall']:.4f}")
print(f"Accuracy: {metrics2['accuracy']:.4f}")
print(f"F1-Score: {metrics2['f1']:.4f}")

# مقایسه میانگین‌ها برای مثال 2
means2 = compare_means(metrics2['precision'], metrics2['recall'])
print(f"\nمقایسه میانگین‌ها:")
print(f"میانگین حسابی: {means2['arithmetic']:.4f}")
print(f"میانگین هندسی: {means2['geometric']:.4f}")
print(f"میانگین هارمونیک: {means2['harmonic']:.4f}")

# کاربر حقوقی (Recall مثال 3: سناریوی اولویت با
print("\n" + "=" * 60)
print("(کاربر حقوقی) Recall مثال 3: سناریوی اولویت با")
print("=" * 60)
tp3, fp3, fn3, tn3 = 4, 100, 1, 895 # 4 سند مرتبط از دست رفته 1 سند نامرتب بازبایی شده، 100 سند مرتبط بازبایی شده،
metrics3 = calculate_ir_metrics(tp3, fp3, fn3, tn3)
print(f"Precision: {metrics3['precision']:.4f}")
print(f"Recall: {metrics3['recall']:.4f}")
print(f"Accuracy: {metrics3['accuracy']:.4f}")
print(f"F1-Score: {metrics3['f1']:.4f}")

# برای beta مقادیر مختلف F-beta محاسبه
print(f"\nF-beta scores:")
for beta in [0.5, 1, 2]:
    f_beta = calculate_f_beta(metrics3['precision'], metrics3['recall'], beta)
    print(f"F{beta}: {f_beta:.4f}")

# کاربر وب (Precision مثال 4: سناریوی اولویت با
print("\n" + "=" * 60)
print("(کاربر وب) Precision مثال 4: سناریوی اولویت با")
print("=" * 60)
tp4, fp4, fn4, tn4 = 8, 2, 12, 978 # 8 سند مرتبط از دست رفته 12 سند نامرتب بازبایی شده، 2 سند مرتبط بازبایی شده،
metrics4 = calculate_ir_metrics(tp4, fp4, fn4, tn4)
print(f"Precision: {metrics4['precision']:.4f}")
print(f"Recall: {metrics4['recall']:.4f}")
print(f"Accuracy: {metrics4['accuracy']:.4f}")
print(f"F1-Score: {metrics4['f1']:.4f}")

# برای beta مقادیر مختلف F-beta محاسبه
print(f"\nF-beta scores:")
for beta in [0.5, 1, 2]:
    f_beta = calculate_f_beta(metrics4['precision'], metrics4['recall'], beta)
    print(f"F{beta}: {f_beta:.4f}")

# رسم نمودارها
print("\n" + "=" * 60)
print("رسم نمودار مقایسه میانگین‌ها")
print("=" * 60)
plot_means_comparison()

print("\n" + "=" * 60)
print("F-beta scores رسم نمودار")
print("=" * 60)
plot_f_beta_scores()

```

=====

مثال 1: سناریوی متعادل

=====

Precision: 0.4444
Recall: 0.4000
Accuracy: 0.9780
F1-Score: 0.4211

مقایسه میانگین‌ها:

میانگین حسابی: 0.4222
میانگین هندسی: 0.4216
میانگین هارمونیک: 0.4211

=====

Accuracy مشکل معیار) مثال 2: سناریوی نامتوازن

=====

Precision: 0.0000
Recall: 0.0000
Accuracy: 1.0000
F1-Score: 0.0000

مقایسه میانگین‌ها:

میانگین حسابی: 0.0000
میانگین هندسی: 0.0000
میانگین هارمونیک: 0.0000

=====

Recall (کاربر حقوقی) مثال 3: سناریوی اولویت با

=====

Precision: 0.0385
Recall: 0.8000
Accuracy: 0.8990
F1-Score: 0.0734

F-beta scores:

F0.5: 0.0475
F1: 0.0734
F2: 0.1613

=====

Precision (کاربر وب) مثال 4: سناریوی اولویت با

=====

Precision: 0.8000
Recall: 0.4000
Accuracy: 0.9860
F1-Score: 0.5333

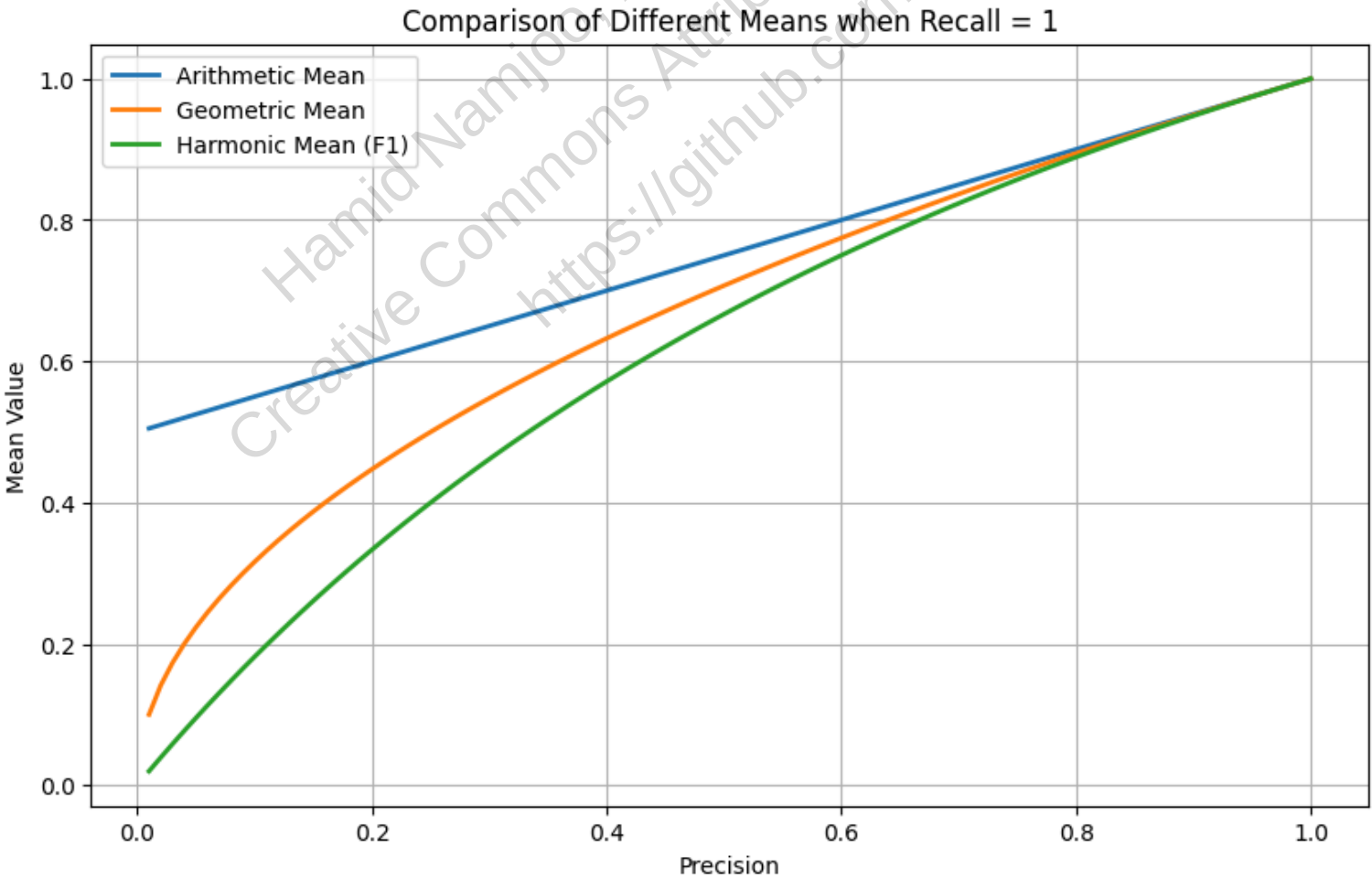
F-beta scores:

F0.5: 0.6667
F1: 0.5333
F2: 0.4444

=====

رسم نمودار مقایسه میانگین‌ها

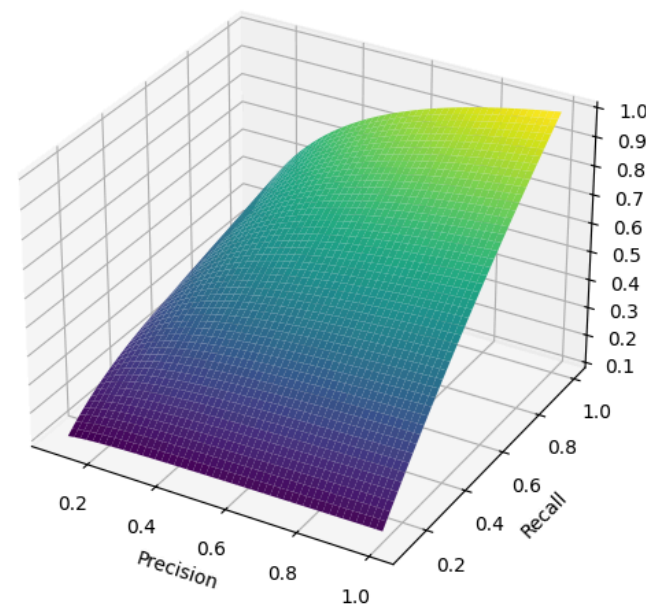
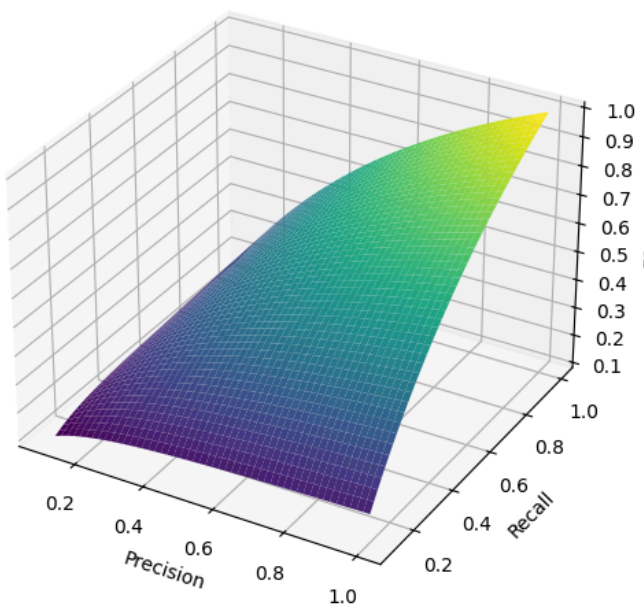
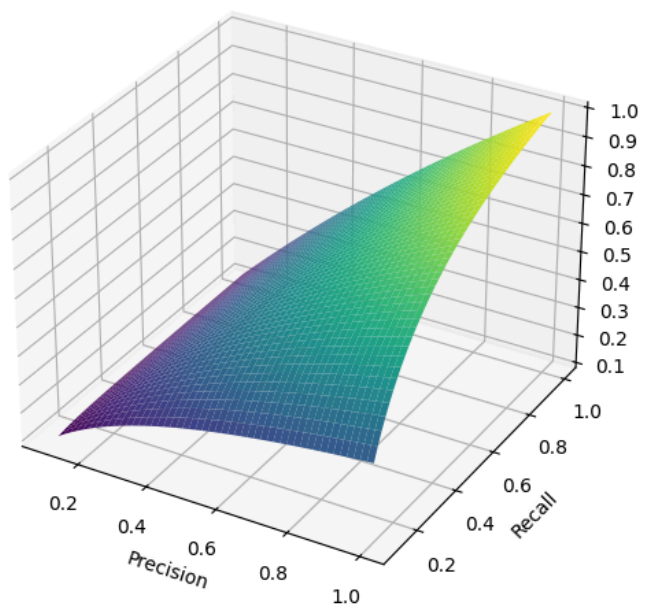
=====



=====

F-beta scores رسم نمودار

=====



۸.۴ ارزیابی نتایج بازیابی رتبه‌دار

وقتی در گوگل عبارت «بهترین رستوران‌های ایتالیایی در تهران» را جستجو می‌کنید، نتایج به صورت رتبه‌دار نمایش داده می‌شوند. اما چگونه می‌توانیم بفهمیم که الگوریتم رتبه‌بندی گوگل چقدر خوب عمل می‌کند؟ معیارهای سنتی مانند precision و recall برای مجموعه‌های بدون رتبه طراحی شده‌اند، اما در دنیای واقعی، ما با نتایج رتبه‌دار سر و کار داریم.

منحنی دقت-بازخوانی (Precision-Recall Curve)

فرض کنید تیم تحقیقاتی در اسنپ در حال ارزیابی الگوریتم جستجوی رستوران‌ها هستند. برای هر پرسمان، می‌توانند precision و recall را در سطوح مختلف k (تعداد رستوران‌های نمایش داده شده) محاسبه کنند. این منحنی معمولاً شکل دنداندار دارد: اگر رستوران بعدی در لیست نامرتب باشد، recall ثابت می‌ماند ولی precision کاهش می‌یابد؛ اگر مرتبط باشد، هر دو افزایش می‌یابند.

برای حذف نوسانات این منحنی، از دقت درونیابی‌شده (Interpolated Precision) استفاده می‌شود:

$$p_{interp}(r) = \max_{r' \geq r} p(r')$$

این فرمول نشان می‌دهد که کاربران حاضرند چند رستوران بیشتر را ببینند اگر این کار درصد رستوران‌های مرتبط در نتایج را افزایش دهد.

نمونه‌ای از ۱۱ سطح interpolated precision برای ارزیابی الگوریتم جستجوی اسنپ:

Interpolated Precision	Recall
1.00	0.0
0.67	0.1
0.63	0.2
0.55	0.3
0.45	0.4
0.41	0.5
0.36	0.6
0.29	0.7
0.13	0.8

میانگین میانگین دقت (Mean Average Precision - MAP)

در مسابقات TREC (مهم‌ترین رویداد ارزیابی سیستم‌های بازیابی اطلاعات)، معیار MAP به عنوان استاندارد طلایی برای ارزیابی سیستم‌ها شناخته می‌شود. MAP میانگین precision در نقاطی که سند مرتبط بازیابی شده است را برای هر پرسمان محاسبه می‌کند و سپس میانگین آن‌ها را روی کل پرسمان‌ها می‌گیرد:

$$MAP(Q) = \frac{1}{|Q|} \sum_{j=1}^{|Q|} \left(\frac{1}{m_j} \sum_{k=1}^{m_j} Precision(R_{jk}) \right)$$

برای مثال، وقتی دیجی‌کالا الگوریتم جستجوی محصولات خود را ارزیابی می‌کند، MAP نشان می‌دهد که به طور متوسط، چقدر محصولات مرتبط در رتبه‌های بالاتر قرار گرفته‌اند. این معیار به خصوص برای کاربرانی که به دنبال خرید محصولات خاص هستند، بسیار مهم است.

دقت در k نتیجه اول (Precision@k)

برای کاربران عادی، مهم‌ترین بخش نتایج جستجو، صفحه اول است. Precision@k دقت را در میان k سند اول اندازه‌گیری می‌کند. برای مثال، Precision@10 نشان می‌دهد که چند درصد از 10 نتیجه اول جستجو مرتبط هستند. این معیار برای شرکت‌هایی مانند گوگل و یاهو که اکثر کاربرانشان هرگز به صفحه دوم نتایج نمی‌روند، بسیار حیاتی است. با این حال، این معیار ناپایدار است و به تعداد کل اسناد مرتبط بستگی دارد.

R-Precision

یک جایگزین برای Precision@10، R-Precision است که دقت را در میان |Rel| سند اول اندازه‌گیری می‌کند، که در آن |Rel| تعداد اسناد مرتبط شناخته شده است:

$$R-Precision = \frac{r}{|Rel|} \quad \text{where } r = \text{number of relevant documents in top } |Rel|$$

این معیار که توسط محققان **موسسه تحقیقاتی مایکروسافت** معرفی شد، مشکل وابستگی به تعداد اسناد مرتبط را حل می‌کند. برای مثال، اگر برای پرسمان «بهترین دوربین‌های حرفه‌ای» 20 سند مرتبط وجود داشته باشد، R-Precision دقت را در 20 نتیجه اول اندازه‌گیری می‌کند.

منحنی ROC (ROC Curve)

منحنی ROC نرخ مثبت واقعی (recall) را در برابر نرخ مثبت کاذب (1 - specificity) رسم می‌کند. در بازیابی اطلاعات کلاسیک، specificity تقریباً همیشه نزدیک ۱ است و نمودار ROC فشرده می‌شود. اما در سیستم‌های مدرن مانند **فیلتر اسپم گوگل**، این معیار بسیار مفید است.

برای مثال، وقتی جیمیل در حال یادگیری تشخیص ایمیل‌های اسپم است، منحنی ROC نشان می‌دهد که چگونه الگوریتم با تغییر آستانه تشخیص، بین تشخیص صحیح ایمیل‌های اسپم و علامت‌گذاری اشتباه ایمیل‌های مهم تعادل برقرار می‌کند.

NDCG (Normalized Discounted Cumulative Gain)

در بسیاری از سیستم‌های مدرن، مرتبط بودن یک سند یک مفهوم دودویی (مرتبط/نامرتبط) نیست، بلکه دارای درجات مختلفی است. برای مثال، در **نتفلیکس**، یک فیلم می‌تواند «کاملاً مطابق سلیقه»، «تا حدی مطابق» یا «اصلاً مطابق» سلیقه کاربر باشد.

NDCG برای چنین سناریوهایی طراحی شده است:

$$NDCG(Q, k) = \frac{1}{|Q|} \sum_{j=1}^{|Q|} \frac{1}{Z_{kj}} \sum_{m=1}^k \frac{2^{R(j,m)} - 1}{\log_2(1 + m)}$$

که در آن Z_{kj} عامل نرمال‌سازی است تا NDCG کامل برابر ۱ شود. این معیار که توسط محققان **یاهو** و **گوگل** توسعه داده شد، امروزه استاندارد طلایی برای ارزیابی سیستم‌های توصیه‌گر است.

برای مثال، وقتی **اسپاتیفای** در حال ارزیابی الگوریتم توصیه موسیقی خود است، NDCG نشان می‌دهد که چقدر موسیقی‌های کاملاً مرتبط در رتبه‌های بالاتر قرار گرفته‌اند. این معیار به طور خاص برای کاربرانی است که به دنبال کشف موسیقی‌های جدید و متناسب با سلیقه خود هستند.

In [8]: `import numpy as np
import math`

```
# =====
# تعریف توابع کمکی برای محاسبه معیارهای بازایی رتبه‌دار
# =====

def calculate_dcg(relevance_scores):
    """
    محاسبه Cumulative Discounted Gain (DCG).
    این تابع، مرتبط بودن نتایج (امتیازدهی شده) را در رتبه‌های مختلف جمع می‌کند و به رتبه‌های پایین‌تر، تخفیف می‌دهد.
    فرمول DCG: Sum_{m=1}^{k} (2^{R(m)} - 1) / log2(1 + m)
    """
    dcg_value = 0.0
    for m, score in enumerate(relevance_scores):
        # m است (m+1) از 0 شروع می‌شود، پس رتبه واقعی m
        # m دارد log2(1 + m)، استفاده می‌کنیم چون صورت m+2 در اینجا از
        # از 1 شروع می‌شود rank که log2(1 + rank)
        # rank = m + 1 رتبه را از 0 نشان می‌دهد، پس m، در کد
        rank = m + 1
        gain = 2**score - 1
        # تنزیل (Discounting): از log2(1 + rank) استفاده
        discount = math.log2(1 + rank)
        dcg_value += gain / discount
    return dcg_value

def calculate_ndcg(retrieved_scores, k):
    """
    نتیجه اول k در Normalized Discounted Cumulative Gain (NDCG) محاسبه
    یک معیار استاندارد برای ارزیابی نتایج یا مرتبط بودن درج‌بندی شده است NDCG
    """
    # 1. برای نتایج واقعی بازایی شده DCG محاسبه
    dcg_val = calculate_dcg(retrieved_scores[:k])

    # 2. بهترین حالت ممکن با مرتب‌سازی نتایج بر اساس مرتبط بودن - Ideal DCG (IDCG) محاسبه
    ideal_scores = sorted(retrieved_scores, reverse=True)
    ideal_dcg_val = calculate_dcg(ideal_scores[:k])

    # 3. نرمال‌سازی: DCG / IDCG
    if ideal_dcg_val == 0:
        return 0.0
    return dcg_val / ideal_dcg_val

def calculate_average_precision(retrieved_labels):
    """
    برای یک پرسمان Average Precision (AP) محاسبه
    میانگین دقت‌ها را در تمام نقاطی که یک سند مرتبط بازایی شده است، محاسبه می‌کند AP
    """
    relevant_count = 0
    precision_sum = 0.0
    total_relevant_docs = retrieved_labels.count('R')

    if total_relevant_docs == 0:
        return 0.0

    for i, label in enumerate(retrieved_labels):
        rank = i + 1
        if label == 'R':
            relevant_count += 1
            # Precision در این رتبه (R_{jk})
            precision_at_rank = relevant_count / rank
            precision_sum += precision_at_rank

    # AP = (مجموع دقت در نقاط مرتبط) / (تعداد کل اسناد مرتبط)
    return precision_sum / total_relevant_docs

# =====
# داده‌های نمونه برای ارزیابی
# =====
```

```
(Non-relevant) برای نامرتب 'N' و (Relevant) برای مرتب 'R': نتایج بازبایی شده #
# فرض کنید الگوریتم 10 نتیجه اول را برگردانده است
retrieved_labels_q1 = ['R', 'N', 'R', 'N', 'N', 'R', 'N', 'N', 'R', 'N']
retrieved_labels_q2 = ['N', 'R', 'R', 'N', 'N', 'N', 'N', 'R', 'N', 'N']
all_queries_labels = [retrieved_labels_q1, retrieved_labels_q2]

# کاملاً مرتب، 0 = نامرتب = 3 (NDCG برای) مرتبط بودن درجه‌بندی شده
relevance_scores = [3, 0, 2, 0, 1, 0, 0, 0, 2, 1]
# رتبه‌هایی که در آنها سند مرتبط وجود دارد (برای مثال اول): 1, 3, 6, 9
# در مثال دوم: 2, 3, 8
relevant_ranks_q1 = [1, 3, 6, 9]
total_relevant_q1 = len(relevant_ranks_q1) # 4 1 پرسمان

# =====
# ۱. Precision@k (نتیجه اول k دقت در)
# =====
print("--- 1. Precision@k (نتیجه اول k دقت در) ---")
k_5 = 5
k_10 = 10

# مرتبط 2 => R, N, R, N, N (q1): نتایج مرتبط در 5 نتیجه اول: P@5 محاسبه
p_at_5 = retrieved_labels_q1[:k_5].count('R') / k_5
print(f"Precision@5 (q1): {p_at_5:.4f} (از 5 نتیجه اول {retrieved_labels_q1[:k_5].count('R')} مورد مرتبط است)")

# مرتبط 4 (q1): نتایج مرتبط در 10 نتیجه اول: P@10 محاسبه
p_at_10 = retrieved_labels_q1[:k_10].count('R') / k_10
print(f"Precision@10 (q1): {p_at_10:.4f} (از 10 نتیجه اول {retrieved_labels_q1[:k_10].count('R')} مورد مرتبط است)")

# =====
# ۲. R-Precision
# =====
print("\n--- 2. R-Precision (دقت در تعداد کل اسناد مرتبط) ---")

# است (تعداد کل اسناد مرتبط برای پرسمان 1) = 4 |Rel| در اینجا
r_precision_q1 = retrieved_labels_q1[:total_relevant_q1].count('R') / total_relevant_q1
print(f"R-Precision (q1, R=|Rel|=4): {r_precision_q1:.4f} (از 4 نتیجه اول {retrieved_labels_q1[:total_relevant_q1].count('R')} مورد مرتبط است)")

# =====
# ۳. Mean Average Precision (MAP) - میانگین میانگین دقت
# =====
print("\n--- 3. Mean Average Precision (MAP) ---")

# برای پرسمان 1 AP محاسبه
ap_q1 = calculate_average_precision(retrieved_labels_q1)
print(f"Average Precision (AP) for Query 1: {ap_q1:.4f}")

# برای پرسمان 2 AP محاسبه
ap_q2 = calculate_average_precision(retrieved_labels_q2)
print(f"Average Precision (AP) for Query 2: {ap_q2:.4f}")

# هاست AP میانگین این MAP
map_score = (ap_q1 + ap_q2) / len(all_queries_labels)
print(f"Mean Average Precision (MAP) for 2 Queries: {map_score:.4f}")

# =====
# ۴. NDCG@k (Normalized Discounted Cumulative Gain)
# =====
print("\n--- 4. NDCG@k (با استفاده از مرتبط بودن درجه‌بندی شده) ---")
k_ndcg = 10 # رتبه تا ارزیابی

# امتیازات مرتبط بودن: [3, 0, 2, 0, 1, 0, 0, 0, 2, 1]
print(f"Relevance Scores (امتیاز مرتبط بودن): {relevance_scores}")

# DCG محاسبه
dcg_value = calculate_dcg(relevance_scores[:k_ndcg])
print(f"DCG@{k_ndcg}: {dcg_value:.4f}")

# NDCG محاسبه
ndcg_score = calculate_ndcg(relevance_scores, k_ndcg)
print(f"NDCG@{k_ndcg}: {ndcg_score:.4f} (مقدار نرمال شده‌ای که به 1 نزدیکتر باشد، بهتر است)")

# =====
# ۵. Interpolated Precision (دقت درونیابی شده)
# =====
# در یک منحنی هستند Precision/Recall داده‌های زیر، نمونه‌ای از نقاط
print("\n--- 5. Interpolated Precision (دقت درونیابی شده) ---")
recall_points = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
# فرض کنید مقادیر دقت در این نقاط بازخوانی به دست آمده‌اند
precision_at_recall = [0.8, 0.75, 0.7, 0.5, 0.4, 0.6, 0.3, 0.15, 0.1, 0.05]

interpolated_precision_values = []
# p_interp(r) = max_{r' >= r} p(r')
for i, r in enumerate(recall_points):
    # max_{r' >= r} p(r') از نقطه کنونی تا انتهای لیست، حداکثر دقت را پیدا کن
    max_precision = max(precision_at_recall[i:])
    interpolated_precision_values.append(max_precision)

print("Interpolated Precision at Standard Recall Points:")
for r, p_interp, p_actual in zip(recall_points, interpolated_precision_values, precision_at_recall):
    print(f"Recall {r:.1f} | Actual P: {p_actual:.4f} | Interpolated P: {p_interp:.4f}")

print("\n(دقت درونیابی شده در واقع حداکثر دقت برای هر سطح بازخوانی کنونی و تمام سطوح بازخوانی بالاتر است)")
```


--- 1. Precision@k (نتیجه اول k دقت در) ---
Precision@5 (q1): 0.4000 (2 مورد مرتبط است)
Precision@10 (q1): 0.4000 (4 مورد مرتبط است)

--- 2. R-Precision (دقت در تعداد کل اسناد مرتبط) ---
R-Precision (q1, R=|Rel|=4): 0.5000 (2 مورد مرتبط است)

--- 3. Mean Average Precision (MAP) ---
Average Precision (AP) for Query 1: 0.6528
Average Precision (AP) for Query 2: 0.5139
Mean Average Precision (MAP) for 2 Queries: 0.5833

--- 4. NDCG@k (با استفاده از مرتبط بودن درجه‌بندی شده) ---
Relevance Scores (امتیاز مرتبط بودن): [1, 2, 0, 0, 0, 1, 0, 2, 0, 3]
DCG@10: 10.0790
NDCG@10: 0.8991 (مقدار نرمال شده‌ای که به 1 نزدیکتر باشد، بهتر است)

--- 5. Interpolated Precision (دقت درونیابی شده) ---
Interpolated Precision at Standard Recall Points:
Recall 0.1 | Actual P: 0.8000 | Interpolated P: 0.8000
Recall 0.2 | Actual P: 0.7500 | Interpolated P: 0.7500
Recall 0.3 | Actual P: 0.7000 | Interpolated P: 0.7000
Recall 0.4 | Actual P: 0.5000 | Interpolated P: 0.6000
Recall 0.5 | Actual P: 0.4000 | Interpolated P: 0.6000
Recall 0.6 | Actual P: 0.6000 | Interpolated P: 0.6000
Recall 0.7 | Actual P: 0.3000 | Interpolated P: 0.3000
Recall 0.8 | Actual P: 0.1500 | Interpolated P: 0.1500
Recall 0.9 | Actual P: 0.1000 | Interpolated P: 0.1000
Recall 1.0 | Actual P: 0.0500 | Interpolated P: 0.0500

(دقت درونیابی شده در واقع حداکثر دقت برای هر سطح بازخوانی کنونی و تمام سطوح بازخوانی بالاتر است)

۸.۵ ارزیابی مرتبط بودن

تصور کنید تیمی در **گوگل** در حال بهبود الگوریتم جستجوی پزشکی خود هستند. برای ارزیابی معتبر این سیستم، نیازهای اطلاعاتی باید با مجموعه‌ی اسناد تست هم‌راستا و متناسب با کاربرد پیش‌بینی‌شده‌ی سیستم باشند. طراحی این نیازها معمولاً توسط متخصصان حوزه انجام می‌شود. استفاده از ترکیب تصادفی واژه‌ها به‌عنوان پرسمان، معمولاً منجر به نتایج غیرواقعی و نامعتبر می‌شود.

مثال واقعی: طراحی پرسمان‌های پزشکی 🔍

در یک پروژه تحقیقاتی در **مایکروسافت ریسرچ**، متخصصان پزشکی پرسمان‌های واقعی بیماران را برای ارزیابی سیستم جستجوی اطلاعات پزشکی طراحی کردند. به جای استفاده از ترکیبات تصادفی مانند "تب سرد قرمز"، از پرسمان‌های واقعی مانند "علائم سندرم رینود در بیماران دیابتی" استفاده کردند که با نیازهای واقعی کاربران مطابقت داشت.

پس از تعریف نیازهای اطلاعاتی، باید ارزیابی مرتبط بودن اسناد نسبت به هر نیاز انجام شود. این فرآیند انسانی، پرهزینه و زمان‌بر است. در مجموعه‌های کوچک مانند Cranfield، قضاوت کامل برای هر زوج پرسمان-سند انجام شده است. اما در مجموعه‌های بزرگ، معمولاً فقط برای زیرمجموعه‌ای از اسناد ارزیابی انجام می‌شود.

روش رایج برای انتخاب این زیرمجموعه، **Pooling** است: اسناد انتخاب‌شده شامل k سند برتر از چند سیستم مختلف، نتایج جست‌وجوی بولی، یا اسناد انتخاب‌شده توسط جست‌وجوگران خبره هستند.

Pooling در عمل: مسابقات TREC 💡

در مسابقات سالانه **TREC** (Text REtrieval Conference)، که توسط NIST سازمان ملی استانداردها و فناوری آمریکا برگزار می‌شود، از روش pooling استفاده می‌شود. هر تیم شرکت‌کننده ۱۰۰۰ سند برتر خود را برای هر پرسمان ارائه می‌دهد. سپس ارزیابان فقط این اسناد را از نظر مرتبط بودن بررسی می‌کنند. این روش باعث صرفه‌جویی چشمگیر در زمان و هزینه می‌شود، زیرا در مجموعه‌های بزرگ مانند GOV2 با ۲۵ میلیون سند، بررسی تمام اسناد غیرممکن است.

ارزیابی انسانی ذاتاً متغیر و فردی است. انسان‌ها دستگاهی نیستند که قضاوت قطعی و یکنواخت ارائه دهند. اما موفقیت یک سیستم IR دقیقاً در توانایی آن برای پاسخ‌گویی به همین نیازهای فردی نهفته است.

مثال حقوقی: تفاوت در قضاوت

در یک مطالعه در شرکت حقوقی کاردان، دو وکیل متخصص اسناد یک پرونده حقوقی را از نظر مرتبط بودن با یک پرسمان خاص ارزیابی کردند. یک وکیل سندی را کاملاً مرتبط دانست زیرا حاوی استناد به یک قانون خاص بود، در حالی که وکیل دیگر همان سند را تنها تا حدی مرتبط ارزیابی کرد زیرا معتقد بود که استناد به قانون در زمینه متفاوتی صورت گرفته است. این مثال نشان می‌دهد که حتی متخصصان یک حوزه ممکن است در مورد مرتبط بودن یک سند اختلاف نظر داشته باشند.

با این حال، بررسی میزان توافق بین ارزیاب‌ها اهمیت دارد. معیار رایج در علوم اجتماعی برای سنجش توافق، آمار کاپا (κ) است که نرخ توافق واقعی را با نرخ توافق تصادفی اصلاح می‌کند:

$$\kappa = \frac{P(A) - P(E)}{1 - P(E)}$$

که در آن:

- $P(A)$: نسبت دفعاتی که دو ارزیاب توافق داشتند
- $P(E)$: نرخ توافق مورد انتظار بر اساس احتمال تصادفی (با استفاده از توزیع‌های حاشیه‌ای)

جدول زیر نمونه‌ای از محاسبه کاپا را نشان می‌دهد:

Judge 2	Yes	No	Total
Judge 1 Yes	300	20	320
Judge 1 No	10	70	80
Total	310	90	400

محاسبات:

- $P(A) = (300 + 70)/400 = 0.925$
- $P(\text{nonrelevant}) = (80 + 90)/800 = 0.2125$
- $P(\text{relevant}) = (320 + 310)/800 = 0.7878$
- $P(E) = 0.2125^2 + 0.7878^2 = 0.665$
- $\kappa = (0.925 - 0.665)/(1 - 0.665) = 0.776$

تفسیر آمار کاپا

مقدار کاپا:

- بالای ۰.۸ ← توافق خوب
- بین ۰.۶۷ تا ۰.۸ ← توافق متوسط
- زیر ۰.۶۷ ← داده مشکوک برای ارزیابی

مطالعه موردی: ارزیابی مرتبط بودن در پزشکی

در یک مطالعه انجام شده در **مرکز تحقیقاتی Mayo Clinic**، دو متخصص ارزیاب اسناد پزشکی را برای ۱۰۰ پرسمان مختلف قضاوت کردند. میانگین آمار کاپا بین آن‌ها ۰.۷۲ بود که در محدوده "توافق متوسط" قرار می‌گیرد. این یافته نشان می‌دهد که حتی متخصصان پزشکی نیز ممکن است در مورد مرتبط بودن یک سند پزشکی اختلاف نظر داشته باشند، که این موضوع طراحی سیستم‌های بازبایی اطلاعات پزشکی را چالش‌برانگیزتر می‌کند.

در TREC و مجموعه‌های پزشکی، توافق معمولاً در محدوده «متوسط» قرار دارد. با وجود تفاوت در قضاوت ارزیاب‌ها، رتبه‌بندی نسبی سیستم‌ها معمولاً پایدار باقی می‌ماند – حتی اگر ارزیابی بر اساس نظر یکی از ارزیاب‌ها انجام شود.

📌 نکته کلیدی پژوهشی

محققان در **دانشگاه کلمبیا** آزمایشی انجام دادند که در آن نتایج ارزیابی را بر اساس قضاوت‌های دو ارزیاب مختلف مقایسه کردند. اگرچه انتخاب ارزیاب می‌توانست تفاوت مطلق در امتیازات را ایجاد کند، اما تأثیر کمی بر رتبه‌بندی نسبی سیستم‌های مختلف داشت. این یافته اهمیت تمرکز بر بهبود نسبی سیستم‌ها به جای تمرکز بر امتیازات مطلق را نشان می‌دهد.

۸.۵.۱ نقد و توجیه مفهوم مرتبط بودن

وقتی مهندسان **گوگل** الگوریتم جدیدی برای جستجو طراحی می‌کنند، چگونه می‌توانند با اطمینان بگویند که این الگوریتم بهتر از نسخه قبلی عمل می‌کند؟ پاسخ در ارزیابی‌های رسمی است. مزیت اصلی ارزیابی سیستم‌های IR با استفاده از مدل استاندارد مرتبط/نامرتب این است که بستری ثابت برای مقایسه سیستم‌ها و پارامترها فراهم می‌کند. این روش بسیار ارزان‌تر از مطالعات کاربری است و امکان بهینه‌سازی خودکار با یادگیری ماشین را فراهم می‌کند.

🔍 مثال واقعی: بهینه‌سازی در میکروسافت

تیم جستجوی **بینگ** در میکروسافت از ارزیابی‌های رسمی برای بهینه‌سازی الگوریتم‌های خود استفاده می‌کند. آنها با استفاده از مجموعه‌های تست استاندارد مانند TREC، می‌توانند تأثیر تغییرات پارامترهای مختلف را به صورت کمی اندازه‌گیری کنند. این رویکرد به آنها اجازه می‌دهد تا با استفاده از یادگیری ماشین، وزن‌های مختلف در معادله امتیازدهی را به صورت خودکار تنظیم کنند، به جای اینکه این کار را به صورت دستی انجام دهند.

البته، اگر معیار رسمی با نیاز واقعی کاربران هم‌راستا نباشد، بهبود آن معیار لزوماً منجر به رضایت کاربر نمی‌شود. در عمل، معیارهای رسمی ساده‌سازی شده‌اند، اما به اندازه کافی خوب هستند. بسیاری از تکنیک‌های موفق مانند نرمال‌سازی طول سند و تنظیم وزن‌ها با یادگیری ماشین، در همین چارچوب توسعه یافته‌اند.

چالش‌های مدل استاندارد ارزیابی

با این حال، این مدل دارای کاستی‌های مفهومی است:

- **استقلال اسناد:** مرتبط بودن یک سند مستقل از سایر اسناد فرض می‌شود. در واقعیت، کاربران مجموعه‌ای از اسناد را بررسی می‌کنند و ارزش یک سند ممکن است به اسناد دیگری که قبلاً دیده‌اند بستگی داشته باشد.
- **قضاوت دودویی:** قضاوت‌ها دودویی هستند (مرتبط یا نامرتبط) و ظرافت ندارند. در واقعیت، یک سند ممکن است "کاملاً مرتبط"، "تا حدی مرتبط" یا "فقط در بخش کوچکی مرتبط" باشد.

- **ذات عینی قضاوت:** مرتبط بودن به صورت تصمیمی مطلق و عینی در نظر گرفته می‌شود، در حالی که در واقعیت قضاوتی ذهنی است که بین افراد مختلف متفاوت است.
- **ثبات نیاز اطلاعاتی:** نیاز اطلاعاتی کاربر در طول جست‌وجو ثابت فرض می‌شود، در حالی که در واقعیت، کاربران با دیدن نتایج، نیاز خود را اصلاح می‌کنند.
- **وابستگی به حوزه:** نتایج به شدت به مجموعهٔ اسناد و پرسمان‌ها وابسته‌اند و ممکن است قابل تعمیم نباشند. سیستمی که در حوزه پزشکی خوب عمل می‌کند، لزوماً در حوزه حقوقی موفق نخواهد بود.

مثال: تفاوت در قضاوت در آمازون

در یک مطالعه در آمازون، دو ارزیاب محصول "دوربین عکاسی حرفه‌ای" را برای پرسمان "بهترین دوربین برای عکاسی ورزشی" ارزیابی کردند. یک ارزیاب دوربینی را که قابلیت فیلم‌برداری با فریم‌ریت بالا داشت کاملاً مرتبط دانست، در حالی که ارزیاب دیگر همان دوربین را تنها تا حدی مرتبط ارزیابی کرد زیرا معتقد بود برای عکاسی ورزشی، سرگشودگی لنز مهم‌تر است. این مثال نشان می‌دهد که حتی متخصصان یک حوزه ممکن است در مورد مرتبط بودن یک سند اختلاف نظر داشته باشند.

برخی از این مشکلات قابل اصلاح‌اند. مثلاً در INEX و برخی مسیرهای TREC و NTCIR، از ارزیابی ترتیبی (ordinal) با چند سطح مرتبط بودن استفاده شده است که اسناد را به دسته‌های "کاملاً مرتبط"، "تا حدی مرتبط" و "نامرتب" تقسیم می‌کند.

مرتبط بودن حاشیه‌ای (Marginal Relevance)

یکی از مشکلات واضح با ارزیابی مبتنی بر مرتبط بودن، تمایز بین مرتبط بودن و مرتبط بودن حاشیه‌ای است: آیا یک سند همچنان دارای ارزش متمایز است پس از اینکه کاربر اسناد دیگری را بررسی کرده است؟ حتی اگر سندی مرتبط باشد، ممکن است اطلاعات آن تکراری با اسناد قبلی باشد.

مثال: تکراری بودن خبرها در گوگل

وقتی در گوگل "نتیجه انتخابات ریاست جمهوری آمریکا" را جستجو می‌کنید، ممکن است ۱۰ نتیجه اول همگی از منابع مختلف (CNN، BBC، نیویورک تایمز و ...) تقریباً همان اطلاعات را گزارش کنند. اگرچه هر کدام از این اسناد به صورت جداگانه مرتبط هستند، اما پس از دیدن اولین سند، ارزش حاشیه‌ای ۹ سند بعدی بسیار کم است. در چنین شرایطی، مرتبط بودن حاشیه‌ای معیار بهتری برای سنجش سودمندی واقعی برای کاربر است.

این موضوع در وب بسیار رایج است (اسناد تکراری یا خلاصه‌های مشابه). بنابراین، معیار بهتر، مرتبط بودن حاشیه‌ای است که تنوع و تازگی را نیز لحاظ می‌کند. حداکثر کردن مرتبط بودن حاشیه‌ای مستلزم بازگرداندن اسنادی است که تنوع و نوآوری را نشان دهند.

برای سنجش مرتبط بودن حاشیه‌ای، می‌توان از «حقایق متمایز» یا «موجودیت‌های منحصربه‌فرد» به عنوان واحد ارزیابی استفاده کرد. این روش شاید مستقیماً سودمندی واقعی را برای کاربر اندازه‌گیری کند، اما ساخت مجموعهٔ تست را دشوارتر می‌کند.

نوآوری در نتفلیکس

در نتفلیکس، وقتی کاربر به دنبال فیلم‌های اکشن می‌گردد، سیستم نه تنها فیلم‌های مرتبط را پیشنهاد می‌دهد، بلکه تلاش می‌کند تنوع را نیز حفظ کند. برای مثال، اگر کاربر چند فیلم ابرقهرمانی دیده باشد، نتفلیکس ممکن است فیلم‌های اکشن با

تم‌های متفاوت (مانند تعقیب و گریز، جنگی یا علمی-تخیلی) را پیشنهاد دهد تا تجربه کاربر را غنی‌تر کند. این رویکرد، تلاشی برای حداکثر کردن مرتبط بودن حاشیه‌ای است.

۸.۶ دید گسترده‌تر: کیفیت سیستم و مطلوبیت کاربر

تا کنون معیارهای رسمی مانند Precision و Recall را برای ارزیابی سیستم‌های بازیابی اطلاعات بررسی کردیم، اما این معیارها فاصله‌ای با نهایت علاقه ما دارند: **چقدر هر کاربر از نتایجی که سیستم برای هر نیاز اطلاعاتی او ارائه می‌دهد، رضایت دارد؟**

مطالعات کاربری در گوگل

گوگل سالانه میلیون‌ها دلار برای مطالعات کاربری هزینه می‌کند. این مطالعات شامل معیارهای کمی مانند **زمان تکمیل یک وظیفه** (مثلاً پیدا کردن اطلاعات و خرید یک محصول) و معیارهای کیفی مانند **امتیاز رضایت** از موتور جستجو و نظرات کاربران در مورد رابط کاربری است. این داده‌ها به گوگل کمک می‌کند تا الگوریتم‌های خود را بهبود بخشد.

در این بخش، به جنبه‌های دیگر سیستم که امکان ارزیابی کمی دارند و مسئله مطلوبیت کاربر را بررسی می‌کنیم.

مسائل سیستم

معیارهای عملی زیادی برای ارزیابی یک سیستم بازیابی اطلاعات فراتر از کیفیت بازیابی وجود دارد:

- **سرعت ایندکس‌گذاری:** سیستم چقدر سریع اسناد را ایندکس می‌کند؟ (مثلاً چند سند در ساعت برای توزیع مشخصی از طول اسناد)

مثال: آمازون وب سرویسز

آمازون وب سرویسز (AWS) برای سرویس CloudSearch خود، سرعت ایندکس‌گذاری را به عنوان یک معیار کلیدی بازاریابی می‌کند. برای مشتریانی که با حجم عظیمی از داده‌ها کار می‌کنند، توانایی ایندکس کردن میلیون‌ها سند در روز یک مزیت رقابتی مهم است.

- **سرعت جستجو:** تأخیر (latency) سیستم به عنوان تابعی از اندازه ایندکس چقدر است؟

مثال: بینگ

بینگ به طور مداوم تأخیر جستجو را در سراسر جهان اندازه‌گیری می‌کند. هر بهبود ۱۰۰ میلی‌ثانیه‌ای در تأخیر جستجو می‌تواند منجر به افزایش قابل توجهی در تعداد جستجوهای روزانه شود، زیرا کاربران ترجیح می‌دهند نتایج را سریع‌تر دریافت کنند.

- **زبان پرسمان:** زبان پرسمان چقدر گویا و قدرتمند است؟ سیستم چقدر با پرسمان‌های پیچیده کار می‌کند؟

مثال: Elasticsearch

Elasticsearch که توسط بسیاری از شرکت‌ها برای جستجوی داخلی استفاده می‌شود، به دلیل زبان پرسمان غنی خود که شامل کوئری‌های پیچیده، فیلترهای تو در تو و تجزیه و تحلیل متن است، محبوب است. این قابلیت به

کاربران امکان می‌دهد تا جستجوهای دقیق‌تری انجام دهند.

- **اندازه مجموعه اسناد:** مجموعه اسناد چقدر بزرگ است و آیا اطلاعات در طیف گسترده‌ای از موضوعات توزیع شده است؟

مثال: ویکی‌پدیا

ویکی‌پدیا نه تنها به دلیل حجم عظیم اسناد خود (میلیون‌ها مقاله)، بلکه به دلیل پوشش گسترده موضوعات از علم تا فرهنگ عامه، یک منبع اطلاعاتی ارزشمند است. این پوشش گسترده باعث می‌شود کاربران بتوانند به طیف وسیعی از نیازهای اطلاعاتی خود پاسخ دهند.

همه این معیارها به جز زبان پرسمان، به راحتی قابل اندازه‌گیری هستند: می‌توانیم سرعت یا اندازه را کمی کنیم. انواع مختلف چک‌لیست‌های ویژگی می‌توانند گویایی زبان پرسمان را نیمه‌دقیق کنند.

۸.۶.۲ سودمندی کاربر

آنچه واقعاً به دنبال آن هستیم، راهی برای سنجش میزان رضایت کلی کاربران است، بر اساس مرتبط بودن نتایج، سرعت سیستم و رابط کاربری. بخشی از این فرآیند، درک توزیع افرادی است که می‌خواهیم آن‌ها را راضی نگه داریم، که این موضوع کاملاً به بستر مورد استفاده بستگی دارد.

مثال: گوگل و وفاداری کاربران

برای یک موتور جستجوی وب مانند گوگل، کاربران راضی کسانی هستند که آنچه را می‌خواهند پیدا می‌کنند. یک معیار غیرمستقیم برای سنجش این رضایت، نرخ بازگشت کاربران به همان موتور جستجو است. گوگل این معیار را به دقت اندازه‌گیری می‌کند، زیرا کاربرانی که به طور مداوم از گوگل استفاده می‌کنند، معمولاً نتایج را مفید می‌یابند. این معیار حتی مؤثرتر می‌شود اگر بتوانیم میزان استفاده کاربران از موتورهای جستجوی دیگر را نیز اندازه‌گیری کنیم.

اما کاربران مدرن موتورهای جستجوی وب فقط کاربران نهایی نیستند؛ **تبلیغ‌دهندگان** نیز کاربران این سیستم‌ها هستند. آن‌ها زمانی راضی هستند که مشتریان روی تبلیغاتشان کلیک کرده و سپس خرید انجام دهند.

مثال: دیجی‌کالا و نرخ تبدیل

در یک وب‌سایت تجارت الکترونیک مانند دیجی‌کالا، کاربر احتمالاً به دنبال خرید چیزی است. بنابراین، می‌توانیم معیارهایی مانند **زمان تا خرید** یا **درصد جستجوکنندگانی که به خریدار تبدیل می‌شوند** را اندازه‌گیری کنیم. دیجی‌کالا این معیارها را به دقت رصد می‌کند تا بفهمد کدام الگوریتم‌های جستجو به بیشترین فروش منجر می‌شوند.

مثال: جستجوی داخلی در شرکت‌ها

برای یک موتور جستجوی داخلی در یک سازمان (شرکت، دولت یا دانشگاه)، معیار مرتبط احتمالاً **بهره‌وری کاربر** است: کاربران چقدر زمان برای پیدا کردن اطلاعاتی که نیاز دارند صرف می‌کنند؟ در مایکروسافت، سیستم جستجوی داخلی بر اساس این

معیار بهینه‌سازی شده است تا کارمندان بتوانند سریع‌تر به اسناد مورد نیاز خود دسترسی پیدا کنند و زمان کمتری برای جستجو صرف کنند.

در حالت کلی، باید تصمیم بگیریم که آیا در تلاش برای بهینه‌سازی، رضایت کاربر نهایی یا صاحب وب‌سایت تجارت الکترونیک را در نظر داریم. معمولاً، صاحب کسب‌وکار است که به ما پول می‌پردازد.

چالش‌های اندازه‌گیری رضایت کاربر

رضایت کاربر اندازه‌گیری مشکل و گریزپذیری است و این یکی از دلایلی است که روش‌شناسی استاندارد از نماینده مرتبط بودن نتایج جستجو استفاده می‌کند. روش مستقیم استاندارد برای دستیابی به رضایت کاربر، اجرای مطالعات کاربری است که در آن افراد در وظایف شرکت می‌کنند، معمولاً معیارهای مختلفی اندازه‌گیری می‌شود، از شرکت‌کنندگان مشاهده می‌شود و از تکنیک‌های مصاحبه قوم‌نگاری برای کسب اطلاعات کیفی در مورد رضایت استفاده می‌شود.

مثال: مطالعات کاربری در نتفلیکس

نتفلیکس به طور منظم مطالعات کاربری انجام می‌دهد تا رضایت کاربران از سیستم توصیه فیلم را ارزیابی کند. در این مطالعات، کاربران وظایفی مانند پیدا کردن فیلم‌های جدید برای تماشا را انجام می‌دهند و زمان تکمیل وظیفه، رضایت از نتایج و تعامل با رابط کاربری اندازه‌گیری می‌شود. این مطالعات به نتفلیکس کمک می‌کند تا الگوریتم‌های توصیه خود را بهبود بخشد.

مطالعات کاربری در طراحی سیستم بسیار مفید هستند، اما انجام آن‌ها زمان‌بر و پرهزینه است. همچنین انجام خوب این مطالعات دشوار است و برای طراحی مطالعات و تفسیر نتایج به تخصص نیاز است. ما در اینجا جزئیات تست قابلیت استفاده انسانی را مورد بحث قرار نخواهیم داد.

۸.۶.۳ بهبود سیستم‌های مستقر

تصور کنید تیمی در گوگل در حال بهبود الگوریتم جستجوی خود است. با میلیون‌ها کاربر در سراسر جهان، چگونه می‌توانند با اطمینان بگویند که تغییر پیشنهادی واقعاً بهتر عمل می‌کند؟ اینجاست که آزمایش‌های زنده (Live Testing) وارد عمل می‌شوند.

مثال واقعی: تست A/B در گوگل

گوگل به طور مداوم تغییرات کوچکی در الگوریتم جستجوی خود آزمایش می‌کند. برای مثال، آنها ممکن است وزن داده شده به عنوان صفحه (title tag) را در الگوریتم رتبه‌بندی تغییر دهند. سپس ۱٪ از کاربران به صورت تصادفی به نسخه جدید هدایت می‌شوند، در حالی که ۹۹٪ باقی‌مانده از نسخه اصلی استفاده می‌کنند. با اندازه‌گیری نرخ کلیک روی نتایج برتر، می‌توانند با اطمینان بگویند که آیا این تغییر مثبت بوده است یا خیر.

تست A/B: روش استاندارد صنعت

رایج‌ترین روش آزمایش، تست A/B است - اصطلاحی برگرفته از صنعت تبلیغات که در حال حاضر در تمام شرکت‌های فناوری بزرگ استفاده می‌شود. در این روش، فقط یک پارامتر بین سیستم فعلی و نسخه پیشنهادی تغییر می‌کند، و درصد کوچکی از کاربران (معمولاً ۱ تا ۱۰٪) به صورت تصادفی به نسخه جدید هدایت می‌شوند.

آمازون از تست A/B برای بهینه‌سازی صفحه نتایج جستجوی خود استفاده می‌کند. برای مثال، آنها ممکن است ترتیب نمایش محصولات را تغییر دهند: در نسخه A، محصولات بر اساس محبوبیت مرتب می‌شوند، در حالی که در نسخه B، محصولات بر اساس امتیاز کاربران مرتب می‌شوند. سپس نرخ تبدیل (درصد کاربرانی که خرید انجام می‌دهند) در هر دو نسخه اندازه‌گیری می‌شود. اگر نسخه B نرخ تبدیل بالاتری داشته باشد، به تدریج برای همه کاربران اجرا می‌شود.

در این نوع آزمایش‌ها، معیارهای کلیدی معمولاً شامل موارد زیر است:

- **نرخ کلیک روی نتیجه اول:** چه درصدی از کاربران روی اولین نتیجه کلیک می‌کنند؟
- **نرخ کلیک روی هر نتیجه در صفحه اول:** توزیع کلیک‌ها در صفحه اول نتایج چگونه است؟
- **زمان ماندن در صفحه:** کاربران چقدر طولانی در صفحه نتایج باقی می‌مانند؟
- **نرخ تبدیل:** چه درصدی از کاربران اقدام مورد نظر (مانند خرید یا ثبت‌نام) را انجام می‌دهند؟

این روش به تحلیل لاگ کلیک (Clickthrough Log Analysis) معروف است:

$$\text{Clickthrough Rate} = \frac{\text{Number of Clicks}}{\text{Number of Impressions}} \times 100\%$$

مزایا و محدودیت‌های تست A/B

پایه تست A/B اجرای مجموعه‌ای از آزمایش‌های تک متغیره است - هر بار فقط یک پارامتر تغییر می‌کند. این باعث می‌شود اثر هر تغییر به وضوح قابل مشاهده باشد. با داشتن تعداد زیاد کاربر، حتی تغییرات بسیار کوچک نیز قابل اندازه‌گیری هستند.

🎬 مثال: نتفلیکس و آزمایش‌های چندمتغیره

نتفلیکس علاوه بر تست‌های A/B، از آزمایش‌های چند متغیره نیز استفاده می‌کند. برای مثال، آنها ممکن است همزمان چند پارامتر را در الگوریتم توصیه فیلم تغییر دهند: اندازه تصاویر، توضیحات فیلم، و دکمه‌های فراخوان. سپس با استفاده از تحلیل آماری چندمتغیره مانند رگرسیون خطی چندگانه، تأثیر هر تغییر را به صورت جداگانه و ترکیبی ارزیابی می‌کنند.

در اصل، می‌توان با تغییر همزمان چند پارامتر به صورت تصادفی و تحلیل آماری چندمتغیره، قدرت تحلیلی بیشتری به دست آورد. اما در عمل، تست A/B محبوب‌تر است، زیرا:

- **پیاده‌سازی آسان:** نیازی به زیرساختار پیچیده ندارد
- **تفسیر ساده:** نتایج به وضوح قابل درک هستند
- **قابل توضیح برای مدیریت:** می‌توان به سادگی به مدیران توضیح داد که چرا یک تغییر مثبت یا منفی بوده است

📊 مثال: تصمیم‌گیری داده‌محور در فیسبوک

فیسبوک از تست A/B برای بهبود الگوریتم نمایش پست‌ها در فید خبری استفاده می‌کند. با آزمایش تغییرات مختلف و اندازه‌گیری تعامل کاربران، آنها می‌توانند تصمیمات داده‌محور بگیرند. برای مثال، اگر تست نشان دهد که نمایش ویدیوها در بالای فید نرخ تعامل را افزایش می‌دهد، این تغییر به صورت دائمی اجرا می‌شود.

در مجموع، تست A/B ابزاری مؤثر برای بهبود تدریجی سیستم‌های IR در محیط واقعی است که به شرکت‌ها اجازه می‌دهد تا با ریسک کم، تغییرات مؤثری در سیستم‌های خود ایجاد کنند.

۸.۷ بریده‌های نتایج (Results Snippets)

وقتی در گوگل عبارت «اقتصاد پاپوا گینه نو» را جستجو می‌کنید، چه چیزی در صفحه نتایج می‌بینید؟ علاوه بر عنوان و لینک، یک متن کوتاه نمایش داده می‌شود که به شما کمک می‌کند بفهمید این صفحه به پرسمان شما مرتبط است یا نه. این متن کوتاه **بریده (snippet)** نام دارد و هدف آن کمک به کاربر برای تصمیم‌گیری در مورد مرتبط بودن سند است.

مثال واقعی: گوگل و بریده‌های پویا

وقتی شما عبارت «اقتصاد پاپوا گینه نو» را جستجو می‌کنید، گوگل بریده‌هایی را نمایش می‌دهد که عبارت جستجو شده را در متن برجسته می‌کند. این بریده‌ها معمولاً از قسمتی از متن اصلی استخراج می‌شوند و به کاربر کمک می‌کنند تا بفهمند سند حاوی اطلاعات مورد نظر آنهاست یا خیر.

انواع بریده‌ها: ایستا در برابر پویا

دو نوع اصلی بریده وجود دارد: **ایستا (Static)** و **پویا (Dynamic)**. بریده‌های ایستا همیشه یکسان هستند، در حالی که بریده‌های پویا بر اساس پرسمان کاربر سفارشی می‌شوند.

بریده‌های ایستا: معمولاً شامل بخشی از سند یا فراداده‌های مرتبط با سند هستند. ساده‌ترین شکل بریده، دو جمله اول یا ۵۰ کلمه اول سند است. به جای بخش‌های سند، می‌توان از فراداده‌های مرتبط با سند مانند عنوان، نویسنده یا توضیحات متا استفاده کرد.

مثال: بریده‌های ایستا در کتابخانه‌های دیجیتال

در سیستم‌های کتابخانه دیجیتال مانند **پروژه گوتنبرگ**، بریده‌های ایستا معمولاً شامل اطلاعات کتاب‌شناختی مانند عنوان، نویسنده، سال انتشار و خلاصه‌ای از محتوای کتاب است. این اطلاعات در زمان ایندکس‌گذاری استخراج و ذخیره می‌شوند تا در هنگام نمایش نتایج جستجو، سریعاً در دسترس باشند.

بریده‌های پویا: تلاش می‌کنند توضیح دهند چرا سند خاصی برای پرسمان فعلی بازایی شده است. این بریده‌ها معمولاً یک یا چند «پنجره» از سند را نمایش می‌دهند که حاوی کلمات کلیدی پرسمان هستند. به این بریده‌ها «کلمات کلیدی در بافت (Keyword-in-Context)» یا KWIC نیز گفته می‌شود.

مثال: بریده‌های پویا در بینگ

بینگ از تکنیک‌های پردازش زبان طبیعی برای تولید بریده‌های پویا استفاده می‌کند. وقتی شما عبارتی را جستجو می‌کنید، بینگ کلمات کلیدی را در متن برجسته می‌کند و تلاش می‌کند پنجره‌هایی از متن را نمایش دهد که به طور خاص به پرسمان شما پاسخ می‌دهند. این بریده‌ها معمولاً خواناتر و مفیدتر هستند زیرا زمینه را حفظ می‌کنند.

چالش‌های فنی در تولید بریده‌ها

تولید بریده‌های پویا چالش‌های فنی خاصی دارد. اگر سیستم فقط ایندکس موقعیتی دارد، نمی‌تواند به راحتی بافت اطراف برخوردهای جستجو را بازسازی کند. این یکی از دلایل استفاده از بریده‌های ایستا است. راه حل استاندارد در دنیای دیسک‌های بزرگ و ارزان، ذخیره محلی تمام اسناد در زمان ایندکس‌گذاری است.

مثال: بهینه‌سازی در گوگل ⚡

گوگل برای بهینه‌سازی تولید بریده‌ها، فقط پیشنوندهای ثابتی از اسناد را ذخیره می‌کند (مثلاً ۱۰,۰۰۰ کاراکتر). برای اکثر اسناد کوتاه، کل سند ذخیره می‌شود، اما ذخیره محلی عظیم برای اسناد بالقوه بزرگ هدر نمی‌رود. بریده‌های اسنادی که طولشان از اندازه پیشنهاد تجاوزی کند، تنها بر اساس مواد در پیشنهاد خواهد بود که به طور کلی یک منطقه مفید برای جستجوی خلاصه سند است.

اگر سندی از زمان آخرین پردازش توسط ایندکس‌گر به‌روز شده باشد، این تغییرات نه در کش و نه در ایندکس خواهند بود. در این شرایط، نه ایندکس و نه بریده به درستی محتوای فعلی سند را منعکس می‌کنند، اما تفاوت‌های بین بریده و محتوای واقعی سند برای کاربر نهایی آشکارتر خواهد بود.

نکته کلیدی 📌

بریده‌های خوب باید ویژگی‌های زیر را داشته باشند: (۱) حداکثر اطلاعات مفید در مورد بحث کلمات کلیدی در سند، (۲) به اندازه کافی خود-contained برای خوانایی آسان، و (۳) به اندازه کافی کوتاه برای قرار گرفتن در محدودیت‌های فضای معمول برای بریده‌ها.

تولید بریده‌ها باید سریع باشد زیرا سیستم معمولاً برای هر پرسمان تعداد زیادی بریده تولید می‌کند. به جای ذخیره کل سند، معمولاً فقط پیشنوندهای ثابت و با اندازه سخاوند از سند ذخیره می‌شود.

جمع‌بندی فصل ۸: ارزیابی در بازیابی اطلاعات 📊

در این فصل:

- معیارهای کلاسیک ارزیابی مانند accuracy، precision، recall، F-measure را برای بازیابی بدون رتبه بررسی کردیم.
- مفاهیم ارزیابی رتبه‌دار مانند MAP، Precision@k، R-Precision، NDCG و منحنی‌های precision-recall را تحلیل کردیم.
- مفهوم مرتبط بودن و چالش‌های ارزیابی انسانی را با ابزارهایی مانند آمار کاپا (kappa statistic) بررسی کردیم.
- نقدهایی بر مدل مرتبط بودن دودویی و پیشنهادهایی برای ارزیابی ترتیبی و مرتبط بودن حاشیه‌ای ارائه شد.
- ابعاد سیستمی ارزیابی مانند سرعت ایندکس‌سازی، مقیاس‌پذیری، و زبان پرسمان را معرفی کردیم.
- روش‌های سنجش سودمندی کاربر و مطالعات کاربری را مرور کردیم.
- مفهوم تست A/B برای بهبود سیستم‌های مستقر و تحلیل رفتار کاربر را توضیح دادیم.
- نقش قطعه‌های نتایج (snippets) در تجربه کاربر و طراحی آن‌ها با روش‌های static و dynamic را بررسی کردیم.

این فصل پایه ارزیابی کمی و کیفی سیستم‌های IR را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:

- بازخورد ضمنی و یادگیری از رفتار کاربر (فصل ۹)
- مدل‌های احتمالاتی و زبان‌محور برای رتبه‌بندی (فصل ۱۱)
- یادگیری رتبه‌بندی و بهینه‌سازی سیستم‌های IR (فصل ۱۵)

Hamid Namjoo, Amirhosein Hemmati, Ali mojahed
Creative Commons Attribution 4.0 International (CC BY 4.0)
<https://github.com/hrnrxb/IR-Textbook>